Bloque de conocimiento cerrado

Respuestas de selección única y justificaciones (preguntas 5, 10 y 15)

Prueba técnica – Ingeniero de Datos · Conexxion Sura · Octubre 2026

Repositorio: github.com/soquendo61-dotcom/conexxion-sura-data-engineering (docs/01_cuestionario.md)

#	Pregunta (resumen)	Respuesta
1	Función principal de la capa Bronze	**b)** Conservar una copia fiel de la fuente con metadatos de ingesta
2	Qué agrega Delta Lake sobre Parquet	**b)** Transacciones ACID, control de esquema y time travel
3	Objetivo de una carga incremental con watermark	**b)** Identificar datos nuevos o modificados desde la última ejecución exitosa
4	Transformación vs. acción en Spark	**a)** La transformación es lazy y la acción dispara la ejecución
5	Patrón idempotente	**b)** Reejecutar con la misma entrada produce el mismo estado esperado sin duplicar efectos
6	Uso de SCD tipo 2	**b)** Mantener historial de cambios en dimensiones
7	Riesgo de `SELECT *` en producción	**b)** Introduce acoplamiento al esquema y lectura innecesaria
8	Cuándo usar broadcast join	**b)** Una tabla es suficientemente pequeña para distribuirla a los executors
9	Uso principal de Unity Catalog	**b)** Gobierno unificado de acceso, descubrimiento y linaje
10	Qué debe hacer una cuarentena de datos	**b)** Separar registros rechazados con causa y trazabilidad para su tratamiento
11	Objetivo de un data contract	**a)** Definir expectativas explícitas de esquema, semántica, calidad y operación entre productor y consumidor
12	Qué permite el partition pruning	**a)** Leer solo las particiones necesarias según los filtros
13	Estrategia adecuada para secretos	**c)** Usar un gestor de secretos y referencias seguras
14	Qué mide una métrica de frescura	**b)** La diferencia entre el momento esperado y el momento real de disponibilidad
15	De qué depende una promoción a producción	**b)** Pruebas automatizadas, revisión, aprobación y artefactos versionados

Justificaciones (máximo tres líneas)

Pregunta 5 – b) Idempotencia significa que ejecutar N veces el mismo proceso con la misma entrada deja el mismo estado final que ejecutarlo una vez: sin duplicados ni efectos acumulados. Se logra con `MERGE` por clave, `replaceWhere` por partición/periodo y control de archivos ya procesados. Las demás opciones confunden idempotencia con ejecución única, ausencia de fallos o inmutabilidad.

Pregunta 10 – b) La cuarentena no descarta ni corrige: aísla el registro inválido conservando la regla incumplida, la severidad, la fecha, el pipeline y el identificador de ejecución, para que pueda analizarse, corregirse en la fuente y reprocesarse. Descartar en silencio rompe la trazabilidad y detener todo por un registro es desproporcionado.

Pregunta 15 – b) Promover a producción debe ser un proceso reproducible y auditable: el artefacto versionado (commit/tag/bundle) que pasó pruebas automatizadas y revisión de código en DEV y QA es el mismo que se despliega en PROD, previa aprobación. Copias manuales o criterios personales no son reproducibles ni auditables.

Cómo se aplica cada concepto en esta solución

Concepto	Dónde se evidencia en el repositorio
Bronze fiel + metadatos (1)	`src/ingestion/bronze.py` : columnas como texto + `_source_file` , `_file_hash` , `_run_id` , `_fecha_corte` , `_row_hash`
Delta ACID / time travel (2)	Todas las tablas son Delta administradas en Unity Catalog; `DESCRIBE HISTORY` en `notebooks/08_trazabilidad`
Watermark (3)	`ctl_archivos` (hash de archivo) + `ctl_watermarks` ; diseño CDC en `docs/03_arquitectura.md`
Lazy vs. acción (4)	Transformaciones encadenadas y una sola acción de escritura por tabla; sin `collect()` de datasets
Idempotencia (5)	`replaceWhere` por `_file_hash` (Bronze), `MERGE` por clave (Silver), `replaceWhere` por `fecha_corte` (Gold)
SCD2 (6)	`silver_clientes_historia` (`src/transformations/scd2.py`)
Sin `SELECT *` (7)	Selección explícita de columnas definidas en los contratos (`src/contracts`)
Broadcast (8)	`F.broadcast` sobre dimensiones pequeñas (clientes) en `src/transformations/gold.py`
Unity Catalog (9)	`resources/sql/01_gobierno_unity_catalog.sql`
Cuarentena (10)	`silver_cuarentena` (`src/quality/engine.py`)
Data contracts (11)	`src/contracts/definitions.py` + validación de esquema en Bronze/Silver
Pruning (12)	Liquid clustering por `fecha_corte` en Gold, filtros por periodo
Secretos (13)	La solución no usa secretos; diseño con Databricks Secret Scopes / Azure Key Vault
Frescura (14)	`vw_obs_frescura_sla`
Promoción (15)	`.github/workflows/` + `databricks.yml` (targets dev/qa/prod)